

安徽信息工程学院

教 案

2024~2025 学年秋季学期

开 课 学 院	计算机与软件工程学院
教研室(课程组)	人工智能
课 程 代 码	CSE319
课 程 名 称	数据处理与分析 (Python)
授 课 对 象	人工智能 2301 班
主 讲 教 师	胡婷婷
职 称	助教

二〇24 年 8 月

教案

[illegible]

《数据处理与分析（Python）》课程教案（1）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第一章数据分析基本介绍及 Python 安装（第1节+第2节）					
目的、要求（分了解、理解、掌握三个层次） （1）了解数据分析的定义与内涵； （2）掌握编辑器的安装与使用； （3）掌握 Python 基本编程环境搭建；					
重点 掌握 Python 基本编程环境搭建；					
难点 掌握 Python 基本编程环境搭建；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：完成学生笔记本电脑环境搭建					

教 学 后 记	
------------	--

教 学 过 程	板书或旁注
---------	-------

一、数据分析定义与内涵

【引入】什么是数据分析？

【定义】数据分析是指用适当的统计分析方法对收集来的大量数据进行分析，将它们加以汇总和理解并消化，以求最大化地开发数据的功能，发挥数据的作用。数据分析是为了提取有用信息和形成结论而对数据加以详细研究和概括总结的过程。

数据分析的数学基础在 20 世纪早期就已确立，但直到计算机的出现才使得实际操作成为可能，并使得数据分析得以推广。数据分析是数学与计算机科学相结合的产物。

为什么要做数据分析？

数据分析的目的是把隐藏在一大批看来杂乱无章的数据中的信息集中和提炼出来，从而找出所研究对象的内在规律。在实际应用中，数据分析可帮助人们做出判断，以便采取适当行动。数据分析是有组织有目的地收集数据、分析数据，使之成为信息的过程。这一过程是质量管理体系的支持过程。在产品的整个寿命周期，包括从市场调研到售后服务和最终处置的各个过程都需要适当运用数据分析过程，以提升有效性。例如设计人员在开始一个新的设计以前，要通过广泛的设计调查，分析所得数据以判定设计方向，因此数据分析在工业设计中具有极其重要的地位。

二、经典数据分析案例

沃尔玛经典营销案例：啤酒与尿布

“啤酒与尿布”的故事产生于 20 世纪 90 年代的美国沃尔玛超市中，沃尔玛的超市管理人员分析销售数据时发现了一个令人难于理解的现象：在某些特定的情况下，“啤酒”与“尿布”两件看上去毫无关系的商品会经常出现同一个购物篮中，这种独特的销售现象引起了管理人员的注意，经过后续调查发现，这种现象出现在年轻的父亲身上。

在美国有婴儿的家庭中，一般是母亲在家中照看婴儿，年轻的父亲前去超市购买尿布。父亲在购买尿布的同时，往往会顺便为自己购买啤酒，这样就会出现啤酒与尿布这两件看上去不相干的商品经常会出现同一个购物篮的现象。如果这个年轻的父亲在卖场只能买到两件商品之一，则他很有可能会放弃购物而到另一家商店，直到可以一次同时买到啤酒与尿布为止。

沃尔玛发现了这一独特的现象，开始在卖场尝试将啤酒与尿布摆放在相同的区域，让年轻的父亲可以同时找到这两件商品，并很快地完成购物；而沃尔玛超市也可以让这些客户一次购买两件商品、而不

【博思平台】

抢答：生活中有哪些数据分析案例？

【板书】

数据分析定义、目的

【爱课堂】

讨论：数据分析在生活、商业、工业中的价值。

是一件，从而获得了很好的商品销售收入，这就是“啤酒与尿布”故事的由来。

当然“啤酒与尿布”的故事必须具有技术方面的支持。1993 年美国学者 Agrawal 提出通过分析购物篮中的商品集合，从而找出商品之间关联关系的关联算法，并根据商品之间的关系，找出客户的购买行为。艾格拉沃从数学及计算机算法角度提出了商品关联关系的计算方法——Aprior 算法。沃尔玛从上个世纪 90 年代尝试将 Aprior 算法引入到 POS 机数据分析中，并获得了成功，于是产生了“啤酒与尿布”的故事。

三、教学内容

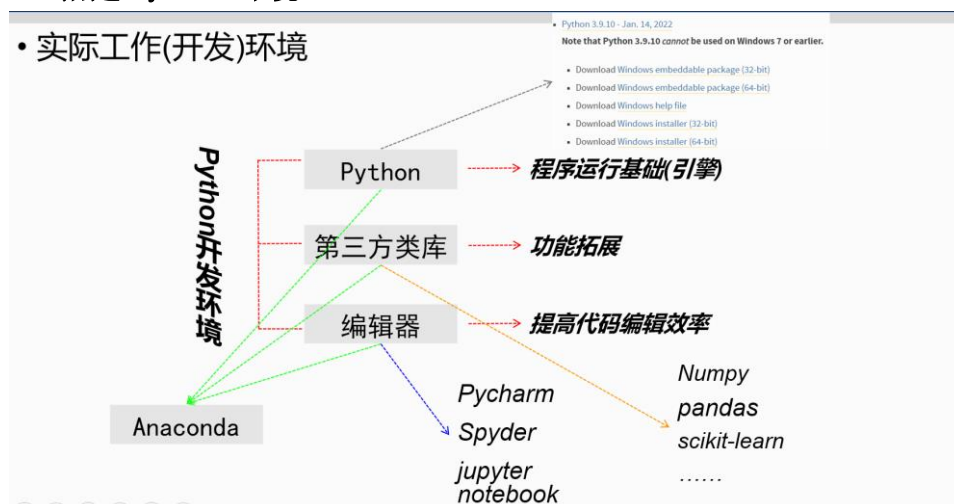
对 Python 起源、优势、应用场景进行介绍

认识 Python



2、搭建 Python 环境

• 实际工作(开发)环境



3、安装 PyCharm

按步骤进行引导 Pycharm 安装。

【任务】
确保学生学会、完成环境安装；

JET
BRAINS

Developer ToolsTeam ToolsLearning ToolsSolutionsStore

PyCharm

What's NewFeaturesLearnBuyDownload



Version: 2021.1
Build: 211.6693.115
7 April 2021

System requirements
Installation Instructions

Download PyCharm

WindowsmacOSLinux

Professional
For both Scientific and Web Python development. With HTML, JS, and SQL support.

Community
For pure Python development

Download
Free trial

Download
Free, open-source

PyCharm

PyCharm Community Edition Setup



Welcome to PyCharm Community Edition Setup

Setup will guide you through the installation of PyCharm Community Edition.

It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.

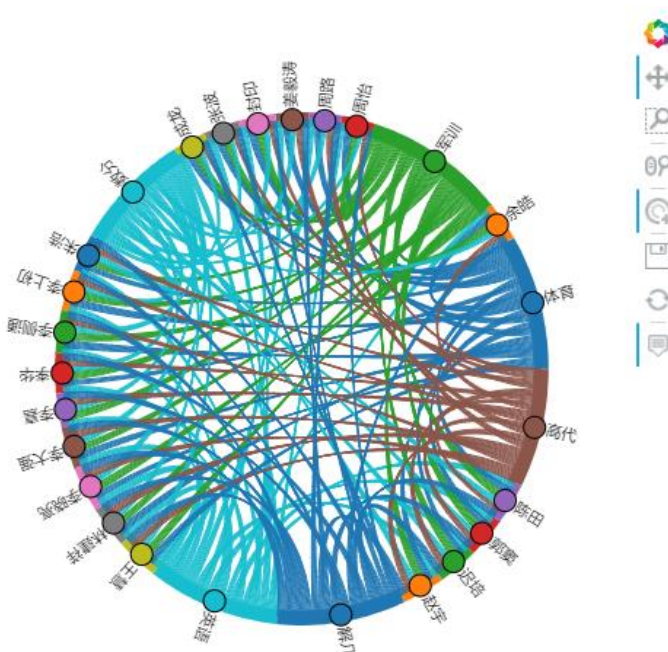
Click Next to continue.

Next >

Cancel

《数据处理与分析（Python）》课程教案（2）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第一章 数据分析基本介绍及 Python 安装（第 3 节+第 4 节）					
目的、要求（分了解、理解、掌握三个层次） （1） 掌握常用 Python 内置函数和模块的导入与使用； （2） Python 基本语法 （3） 了解常见代码错误提示；					
重点 掌握常用 Python 内置函数和模块的导入与使用；					
难点 掌握常用 Python 内置函数和模块的导入与使用；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务： 学生学习常见错误提示： https://zhuanlan.zhihu.com/p/150263797#ModuleNotFoundError%20%E6%A8%A1%E5%9D%97%E4%B8%8D%E5%AD%98%E5%9C%A8					
教 学 后 记					

教 学 过 程	板书或旁注
一、数据分析概览 【引入】数据分析方向 1.Python 语言基础语法 2.Python 数据分析常用类库 3.Python 爬虫 4.数据可视化 matplotlib、epycharts 【引入】数据分析流程 数据获取、数据分析、数据可视化  二、模块和三方库 模块： 模块是一个包含所有你定义的函数和变量的文件，其后缀名是.py # 导入模块 <pre>import math</pre> # 现在可以调用模块里包含的函数了 <pre>math.sin(1.5)</pre> 常用内置模块：math, random, datetime, os, shutil 安装：pip install packagename (numpy, pandas) 导入： 1. import packagename 2. import packagename as new_name 3. from packagename import function_name 调用 1. packagename.function_name 2. new_name.function_name 3. function_name	【板书】 数据分析流程

三、Python 基本语法

Python 基本语法

【任务引入】编写第一个 Python 程序

```
hello_world = 'hello world!'
```

```
print(hello_world)
```

- **注释、缩进；**
- **变量：**标识符就是用于给程序中变量、类、方法命名的符号（简单来说，标识符就是合法的名字）。Python 需要使用标识符给变量命名。Python 中的变量不需要声明，每个变量在使用前都必须赋值，变量赋值以后该变量才会被创建。

标识符必须以字母、下画线（_）开头，后面可以跟任意数目的字母、数字和下画线（_）。

Python 语言是区分大小写的。

标识符不能是 Python 关键字，但可以包含关键字。

标识符不能包含空格。

3、字符串与数值：

【任务引入】

利用 Python 完成以下任务：

- 创建一个字符串变量 “Apple's unit price is 9 yuan.”。
- 提取出里面的数字 9 并赋值给新的变量。
- 查看新变量的数据类型。
- 将提取的数字 9 转成整型（int）。
- 确认数据类型是否转换成功。

字符串创建、基本用法；

任务实现：

1.创建一个字符串变量 “Apple's unit price is 9 yuan.”。

```
applePriceStr = "Apple's unit price is 9 yuan"
```

2.提取出里面的数字 9 并赋值给新的变量。

```
applePrice = applePriceStr[-6]
```

3.查看新变量的数据类型。

```
print(type(applePrice))
```

4.将提取的数字 9 转成整型（int）。

```
applePriceInt = int(applePrice)
```

5.确认数据类型是否转换成功。

```
print(type(applePriceInt))
```

4、操作运算符

【任务引入】

根据相应数学公式，完成以下任务

- 给定圆的半径，计算圆的周长和面积
- 给定圆的周长，计算圆的半径和面积
- 给定圆的面积，计算圆的半径和周长

【板书】
基本语法

【博思】
作业：完成该任务并提交

讲解：算数运算符、比较运算符、赋值运算符、逻辑运算符、成员运算符、身份运算符、运算符优先级；

任务实现：

- `pi = 3.14` # 设置常量
- `r = 3` # 输入圆形的半径
- `C = 2 * pi * r` # 计算圆形的周长
- `S = pi * r ** 2` # 计算圆形的面积
- `print('半径为', r, '的圆形，其周长等于', C, '；面积等于', S, '。')`

- `C = 5` # 输入圆形的周长
- `r = C / (2 * pi)` # 计算圆形的半径
- `S = pi * r ** 2` # 计算圆形的面积
- `print('周长为' + str(C) + '的圆形，其半径为' + str(r) + '；面积等于' + str(S) + '。')`

- `S = 5` # 输入圆形的面积
- `r = round((S / pi) ** 0.5, 2)` # 计算圆形的半径，并保留两位小数
- `C = round(2 * pi * r, 2)` # 计算圆形的周长，并保留两位小数
- `str_print = '面积为' + str(S) + '的圆形，其半径为' + str(r) + '；周长等于' + str(C) + '。'`
- `print(str_print)`

5、基本语法总结：

注释符用井号 #，赋值 str 型引号找；

单引双引无差异，同时使用要注意；

分层次，用空格，冒号之下空四格；

索引序号从 0 起，逆序 - 1 也可以。

四、常见错误提示

1、错误地使用了中文标点符号

报错信息：

SyntaxError: invalid character in identifier

错误示例 1：

```
print('hello', 'world')
```

错误原因：逗号是中文标点符号

2、IndentationError 缩进错误报错信息：

IndentationError: unindent does not match any outer indentation level

IndentationError: expected an indented block

错误示例：

```
a = 2
```

```
while a < 0:
```

```
print('hello')
```

```
a -= 1
```

【博思】

```
else:
```

```
print('0.0')
```

解决方法:

上述代码中 `while` 语句体内的代码缩进没有对齐。正确使用缩进排版代码。当代码是从其它地方复制并粘贴过来的时候，这个错误较多见。

3、NameError 名字错误

当变量名、函数名或类名等书写错误，或者函数在定义之前就被调用等情况下，就会导致名字错误。报错信息：

```
NameError: name 'pirnt' is not defined
```

```
NameError: name 'sayhi' is not defined
```

```
NameError: name 'pd' is not defined
```

4、ModuleNotFoundError 模块不存在

报错信息：

```
ModuleNotFoundError: No module named 'pandas'
```

错误示例 1:

```
import pandas as pd
```

```
# 没有导入成功，报上面错误。
```

解决方法:

这种报错常见于两种场景中，第一、未下载、安装该模块;第二、将调用的模块路径与被调用的模块路径不一致等。第一种情况直接下载安装即可，在 `cmd` 中，`pip install xxx`;第二种情况电脑中可能存在多个版本的 `Python`，建议保留一个常用的即可。

提问：常见错误含义，加深学生理解

《数据处理与分析（Python）》课程教案（3）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第二章 Python 数据结构与流程控制（第1节+第2节）					
目的、要求（分了解、理解、掌握三个层次） （1） 列表、元组、字典、集合等数据结构的异同； （2） 列表、元组、字典、集合的访问、切片、计算等；					
重点 掌握列表、元组、字典、集合等数据结构的异同以及对它们访问、切片、计算等；					
难点 掌握列表、元组、字典、集合等数据结构的异同以及对它们访问、切片、计算等；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：完成代码编写小项目					
教 学 后 记					

教 学 过 程	板书或旁注														
数据结构：根据某种方式将数据元素组合起来形成的一个数据元素集合。 Python 数据结构主要包含序列（如列表和元组）、映射（如字典）和集合 3 种基本的数据结构类型。 一、字符串 1、str 定义 ■ str 就是一串字符，是编程语言中表示文本的数据类型。 ■ 在 Python 中可以使用一对引号（单双引号均可）定义一个字符串。 ■ 可以使用 index 获取一个 str 中指定位置的字符，索引计数从 0 开始。 ■ 也可以使用 for 循环遍历字符串中每一个字符。 2、字符串的增删改查； 3、字符串常用内置方法； 二、列表 1、列表定义 ■ 列表是最常用的数据类型，用方括号表示。 ■ 列表的数据项（元素）不需要具有相同的类型。 ■ 创建一个列表，只要把逗号分隔的不同的元素使用方括号括起来即可 2、列表的增删改查； 3、列表常用内置方法； <table border="1" data-bbox="181 1162 1137 1960"> <thead> <tr> <th>方法名称</th><th>作用说明</th></tr> </thead> <tbody> <tr> <td>list.append(obj)</td><td>在列表末尾添加新的对象</td></tr> <tr> <td>list.count(obj)</td><td>统计某个元素在列表中出现的次数</td></tr> <tr> <td>list.extend(seq)</td><td>在列表末尾一次性追加另一个序列中的多个值（用新列表扩展原来的列表）</td></tr> <tr> <td>list.index(obj)</td><td>从列表中找出某个值第一个匹配项的索引位置</td></tr> <tr> <td>list.insert(index, obj)</td><td>将对象插入列表</td></tr> <tr> <td>list.pop(obj=list[-1])</td><td>移除列表中的一个元素（默认最后一个元素），并且返回该元素的值</td></tr> </tbody> </table>	方法名称	作用说明	list.append(obj)	在列表末尾添加新的对象	list.count(obj)	统计某个元素在列表中出现的次数	list.extend(seq)	在列表末尾一次性追加另一个序列中的多个值（用新列表扩展原来的列表）	list.index(obj)	从列表中找出某个值第一个匹配项的索引位置	list.insert(index, obj)	将对象插入列表	list.pop(obj=list[-1])	移除列表中的一个元素（默认最后一个元素），并且返回该元素的值	
方法名称	作用说明														
list.append(obj)	在列表末尾添加新的对象														
list.count(obj)	统计某个元素在列表中出现的次数														
list.extend(seq)	在列表末尾一次性追加另一个序列中的多个值（用新列表扩展原来的列表）														
list.index(obj)	从列表中找出某个值第一个匹配项的索引位置														
list.insert(index, obj)	将对象插入列表														
list.pop(obj=list[-1])	移除列表中的一个元素（默认最后一个元素），并且返回该元素的值														

list.remove(obj)	移除列表中某个值的第一个匹配项	
4、列表推导式 • a = [] • for i in range(1,11): • a.append(i) • b = [i for i in range(1,11)] • a = [i**2 for i in range(1,10)] • c = [j+1 for j in range(1,10)] 5、列表任务实现 1. 将图形等份划分，得到若干小矩形（构建 x 序列）。 2. 求出各小矩形的高度。 3. 将各高度乘以宽度，得各小矩形面积，最后求和。 三、元组 ■ 元组与列表类似，不同之处在于元组的元素不能修改。 ■ tuple 使用小括号，list 使用方括号。 ■ tuple 创建在括号中添加元素，并使用逗号隔开。 查看 tuple 中的元素同 str 的切片方法。 tuple 好处在于不能被修改，所以 list 可转化为元组浏览，访问比 list 快。 四、集合 ■ 集合（set）是一个无序的不重复元素序列。 ■ 创建集合用 {}，元素用逗号隔开。 ■ 创建一个空集合必须用 set() 而不是 {}，{} 是用来创建一个空字典。 ■ 常用它过滤 list、tuple 等重复值。 Set 是无序的，故没有 index，但要看某元素在不在 set 中，用 in。 五、字典 ■ 字典用于存放有映射关系的两组数据。其一组是 key，是关键数据，具有唯一性，对字典操作都是基于 key；另一组是 value，需通过 key 来访问。 ■ 字典的元素是一对键值，用冒号：分开，字典同 set 都使用花括号 {}。 ■ 字典是一种可变容器模型，可存储任意类型对象。但 key 需为不可变型。 字典的常见增删改查操作； 字典任务： 统计下面这段文字中各单词出现的频次： 'The night begin to shine, the night begin to shine' 六、常见函数 type()：查看数据类型。 len()：查看数据的长度（即含有元素的个数）。		【任务安排】视时间决定课堂引导学生完成或学生课后完成； 【任务安排】视时间决定课堂引导学生完成；

<code>help()</code>: 查看数据的用法。自定义函数的说明文档就是通过 <code>help</code> 查看。 <code>id()</code> : 查看数据存储内存地址。 <code>copy()</code>: 对数据进行复制、拷贝。	
---	--

《数据处理与分析（Python）》课程教案（4）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第二章 Python 数据结构与流程控制（第3节+第4节）					
目的、要求（分了解、理解、掌握三个层次） （1）if 选择、for 循环、while 循环； （2）range 对象在循环中的使用，成员测试符 in 在循环语句中的使用，循环代码的优化； （3）break 和 continue 语句的作用；					
重点 （1）掌握 if 选择、for 循环、while 循环；range 对象在循环中的使用，成员测试符 in 在循环语句中的使用； （2）熟悉 break 和 continue 语句的作用； （3）了解循环代码的优化；					
难点 熟悉 break 和 continue 语句的作用；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
一、流程控制 【引入】什么是数据分析？ 即控制流程,具体指控制程序的执行流程,而程序的执行流程分为三种结构: 顺序结构 分支结构(if) 循环结构(while 与 for) 1、 顺序结构 在 Python 语言中，顺序结构就是按照代码的前后顺序按行执行。 以一元二次方程求解为案例介绍： 对于一元二次方程来说，判断解的个数是根据 $\Delta = b^2 - 4ac$ 的情况判断的，对于如下方程： 本节只讨论存在解的情况（一解的情况可认为两解相同）,程序流程如下： (1) 输入a、b、c (2) 计算Δ (3) 计算解 (4) 输出解 <pre> # 输入a、b、c a = float(input("输入a:")) b = float(input("输入b:")) c = float(input("输入c:")) # 计算delta delta = b**2 - 4 * a * c # 计算解x1、x2 x1 = (b + delta**0.5) / (-2 * a) x2 = (b - delta**0.5) / (-2 * a) # 输出x1、x2 print("x1=", x1) print("x2=", x2) </pre> 代码说明： input()函数是程序接收来自键盘的输入。 如code1-1中，首先要接收来自键盘的输入，即方程的系数a、b、c，才能执行下面代码计算出delta和方程的解。为了给用户一个友好的界面，提醒用户输入，所以我们在input()函数的括号内可以写入一些提示信息，如input("输入A:").当然你也可以写成：input("亲爱的，我在等你输入方程的首系数a呢:"). 2、 选择/判断结构 if 选择结构也叫判断结构或分支结构，根据满足的条件去执行相应的动作，即根据条件判断的真假去执行不同分支所对应的子代码。 <pre> graph TD A[if condition:] -- 成立的条件 --> B[do something] B -- 冒号 --> C[else:] C -- 冒号 --> D[do something] </pre>	【博思平台】 抢答：C 语言中有哪些流程控制？ 【板书】 流程控制



判断结构增加了在程序中的判断机制，对上节的解方程程序，我们可以增加判断结构从而让方程的解更加全面。

本次程序流程如下：

- (1) 输入a、b、c；
- (2) 计算 Δ ；
- (3) 判断解的个数；
- (4) 计算解；
- (5) 输出解。

```
# 输入a、b、c
a = float(input("输入a:"))
b = float(input("输入b:"))
c = float(input("输入c:"))

# 计算delta
delta = b**2 - 4 * a * c

# 判断解的个数
if delta < 0:
    print("该方程无解!")
elif delta == 0:
    x = b / (-2 * a)
    print("x1=x2=", x)
else:
    # 计算x1、x2
    x1 = (b + delta**0.5) / (-2 * a)
    x2 = (b - delta**0.5) / (-2 * a)
    print("x1=", x1)
    print("x2=", x2)
```

3、循环结构

(1) while 循环

while 循环是最简单的循环，几乎所有程序语言中都存在 while 循环这种或者类似结构，while 经典循环结构如下：

while 循环条件为真：

执行块



【课后作业】

成绩等级划分规则：

分数 ≥ 90 ，等级为 A； $80 \leq \text{分数} < 90$ ，等级为 B； $70 \leq \text{分数} < 80$ ，等级为 C； $60 \leq \text{分数} < 70$ ，等级为 D；分数 < 60 ，等级为 E。若输入的内容非成绩分数，则输出错误提示语。

(2) for 循环

for 循环常用来遍历集合，for 循环较 while 循环而言，在程序中用到的会更为普遍。在上节中我们用 while 循环计算了从 1 到任意数的和，本节将用 for 循环完成这个任务。

假设 A 是一个集合，element 代表集合 A 中的元素，for 循环就是每次取 A 中的一个元素 element 都执行一次“循序体”，格式如下：

for element in A:

循环体



(4) 异常值处理

程序在正常执行过程中可能会出现一些异常情况，如语法错误、除零错误、未定义的变量取值等，我们希望程序能够帮我们监控和捕捉到相应的错误。

Python 为我们提供了异常值处理 try 语句，其完整的形式为 try/except/else/finally。

```
try:
    Normal execution block
except A:
    Exception A handle
except B:
    Exception B handle
except:
    Other exception handle
else:
    #可选。若有，则必有except x或except存在，仅在try后无异常执行
    if no exception, get here
finally:
    #此语句可选，但务必放在最后，并且是必须执行的语句
    print("finally")
```

(5) 总结回顾

在 Python 语言中，顺序结构就是按照代码的前后顺序按行执行。

分支结构就是根据满足的条件去执行相应的动作，即根据条件判断的真假去执行不同分支对应的子代码。

循环结构就是重复执行某段代码。

(6) 循环进阶

pass、break、continue;

range 函数、arange 函数、enumerate 函数

《数据处理与分析（Python）》课程教案（5）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第三章 函数设计与使用（第 1 节+第 2 节）					
目的、要求（分了解、理解、掌握三个层次） （1）Python 函数的定义方式； （2）return 语句的使用； （3）变量作用域，局部作用域与全局作用域的区别；					
重点 Python 函数的定义方式					
难点 变量作用域，局部作用域与全局作用域的区别；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
一、函数定义 1、使用 def 定义函数 任务导入： - 使用 def 关键字定义一个求列表均值的自定义函数。 - 列表为：[1,2,6,0.3,2,0.5,-1,2.4] 函数介绍： - 函数实现了对整段程序逻辑的封装 - 从程序代码中独立出来 - 避免出现大段重复代码 - 便于维护 Python 内建函数： <code>print()</code> <code>input()</code> <code>int()</code> 使用 def 关键字定义函数： def function(x,y): return 'result' 自定义函数有固定的格式，它是通过关键字 def 来声明的，其结构如下。 <pre>def 函数名(参数): """ 说明文档内容 """ 函数体 return 返回值</pre> 提示：在函数结构中，一般还需要有一个函数说明文档，放在函数的 def 声明行和函数体之间，文档中主要描述函数的功用以及参数的用法等，便于函数的使用者调用 help()对函数的查询。也就是说，我们用 help()能够查到的帮助文档都是放在函数文档中的。函数文档使用三引号引起来放在函数头和函数体之间。 任务实现： <pre>def mean(x): sum1 = 0 for i in x: sum1 += i return sum1/len(x)</pre> 函数的参数分为形参和实参。形参即形式参数，在使用 def 定义函数时，函数名后面的括号里的变量称作形式参数。在调用函数时提供的值或者变量称作实际参数，实际参数简称为实参。 #这里的 a 和 b 就是形参	【板书】 函数定义

```
def add(a,b):  
    return a+b
```

```
#这里的 1 和 2 是实参  
add(1,2)
```

```
#这里的 x 和 y 是实参  
x=2  
y=3  
add(x,y)
```

《数据处理与分析（Python）》课程教案（6）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第三章 函数设计与使用（第3节+第4节）					
目的、要求（分了解、理解、掌握三个层次） （1）lambda 表达式声明匿名函数，在 lambda 表达式中调用函数； （2）map()、reduce()、filter()的使用；					
重点 （1）lambda 表达式声明匿名函数，在 lambda 表达式中调用函数； （2）map()、reduce()、filter()的使用；					
难点 （1）lambda 表达式声明匿名函数，在 lambda 表达式中调用函数； （2）map()、reduce()、filter()的使用；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
一、使用 lambda 创建匿名函数 lambda 函数又称为匿名函数，或者行内函数。匿名函数多用于调用一次就不再被调用的函数，属于“一次性”函数。 其表达式的语法格式为： <code>lambda para:expr</code> <code>para</code> 为参数，多个参数可以使用逗号隔开，冒号后的 <code>expr</code> 为一个表达式。 匿名函数，没有具体名称的函数 lambda 定义的是单行函数，如果需要复杂的函数，那么应使用 <code>def</code> 关键字。 lambda 函数可以包含多个参数，但 lambda 函数有且只有一个返回值。 <code>example = lambda x : x ** 3</code> 二、数据分析常用内置函数 Map（）： 遍历序列，对序列中每个元素进行同样的操作，最终获取新的序列。 <code>map(f, S)</code> 将函数 <code>f</code> 作用在序列 <code>S</code> 上。 filter()： 对序列中的元素进行筛选，最终获取符合条件的序列。 <code>filter(f, S)</code> 将条件函数 <code>f</code> 作用在序列 <code>S</code> 上，符合条件函数的则输出。 reduce()： 对于序列内所有元素进行累计操作。 <code>reduce(f(x,y), S)</code> 将序列 <code>S</code> 中的第一、第二个数用二元函数 <code>f(x,y)</code> 作用后的结果与第三个数继续用 <code>f(x,y)</code> 作用，再将这个结果与第四个数继续用 <code>f(x,y)</code> 作用，直到最后。 eval()： <code>eval()</code> 函数将字符串 <code>str</code> 当成有效的表达式来求值并返回计算结果，也就是实现 <code>list</code>、<code>dict</code>、<code>tuple</code> 与 <code>str</code> 之间的转化。 三、存储并导入函数模块 任务引入： 将 <code>mean</code> 函数与匿名函数封装成模块，然后导入模块再进行调用。 任务分析： 1. 模块是最高级别的程序组织单元，对应了 Python 的脚本文件（.py） 2. 模块可以被别的程序导入，以使用该模块的函数等功能 3. 导入的函数被当作模块对象的成员被使用	【板书】 函数定义方式 【课堂练习】 学生练习

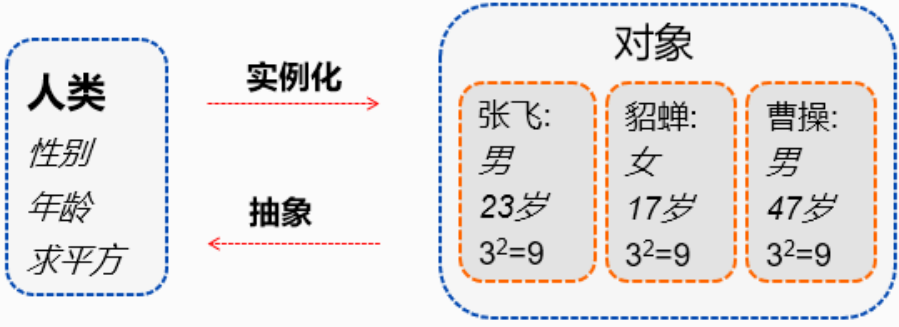
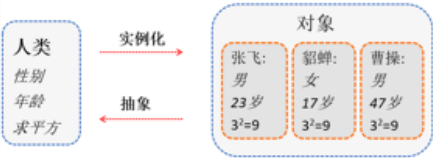
任务实现：

注意被导入模块须位于当前工作路径（一般放置工程文件夹中即可）

from

《数据处理与分析（Python）》课程教案（7）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第四章 面向对象（第1节+第2节）					
目的、要求（分了解、理解、掌握三个层次） （1）面向对象基本概念； （2）类和对象； （3）面相对象基础语法； （4）面向对象封装案例；					
重点 面相对象基础语法；					
难点 面相对象基础语法；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
一、认识面向对象 【引入】面向对象优点？ 大幅减少冗余代码，方便拓展现有代码，提高编码效率，降低了出错概率。 易于维护、质量高。  <pre>class Human: def __init__(self, age, sex): self.age = age self.sex = sex def square(self, x=None): return x**2</pre>  二、面向对象语法 在 Python 中，所有数据类型都可以视为对象，当然也可以自定义对象。自定义的对象 数据类型就是面向对象中的类（Class）的概念。 面向对象最重要的概念就是类（class）和实例（Instance），必须牢记类是抽象的模板，比如 Employee 类，而实例是根据类创建出来的一个个具体的“对象”，每个对象都拥有相同的方法，但各自的数据可能不同。在 Python 中，定义类是通过 class 关键字，以 Employee 类为例 <pre>class Employee(object): pass</pre> class 后面紧接着是类名，即 Employee，类名通常是大写开头的单词，紧接着是 (object)，表示该类是从哪个类继承下来的，关于继承的概念这里不做过多说明，读者可以自行查询资料。通常，如果没有合适的继承类，就使用 object 类，这是所有类最终都会继承的类。 定义好了 Employee 类，就可以根据 Employee 类创建出 Employee 的实例，创建实例是通过“类名 ()”实现的。	【板书】 数据分析定义、目的 【爱课堂】 讨论：数据分析在生活、商业、工业中的价值。

```
In [1]:class Employee(object): #定义一个类
        pass
```

```
In [2]:amy = Employee()          #根据类创建一个类的实例
amy
```

```
Out[2]:
<__main__.Employee at 0x22eaf229208>
```

```
In [3]:Employee
```

```
Out[3]:
```

```
__main__.Employee
```

由于类可以起到模板的作用，因此，可以在创建实例的时候，把一些我们认为必须绑定的属性强制填写进去。通过定义一个特殊的__init__方法，在创建实例的时候，就把 name、salary 等属性绑上去：

```
In [5]:class Employee(object):
        def __init__(self, name, salary):
            self.name = name
            self.salary = salary
```

注意：特殊方法“__init__”前后分别有两个下划线！！

注意到__init__方法的第一个参数永远是 self，表示创建的实例本身，因此在__init__方法内部可以把各种属性绑定到 self，因为 self 就指向创建的实例本身。有了__init__方法，在创建实例时，就不能传入空参数了，必须传入与__init__方法匹配的参数，但 self 不需要传，Python 解释器自己会把实例变量传进去。

```
class House():
    c = "I'm a Newhouse."
    def __init__(self, name, area, room_num):
        self.name = name
        self.area = area
        self.room_num = room_num
        print(House.c, "And my name is %s" % name)
    def room(self):
        print("I'm a room, I'm %d" % self.area)
        e = self.area / self.room_num
        print("My all room is %d" % e)

a = House("新大洲", 150, 5)
a.room()
a.c
```

类名
属性

第一参数self代表的当前的实例对象本身，当然也可以用别的符号代替self，但是这个必须有

方法

实例化对象

面向对象编程有一个重要特点就是数据封装。在上面的 Employee 类中，每个实例就拥有各自的 name 和 salary 这些数据。我们可以通过函数来访问这些数据，比如打印一个员工的工资。

```
In [8]:def print_salary(std):
        print('%s: %s' % (std.name, std.salary))
```

```
In [9]:print_salary(amy)
```

```
Out[9]:
```

Amy Simpson: 59

但是，既然，但要访问这些数据就没有必要从外部的函数去访问，可以直接在 Employee 类的内部定义访问数据 **Employee 实例本身就拥有这些数据的函数**，这样就把“数据”给封装起来了。这些封装数据的函数是和 Employee 类本身是关联起来的，我们称之为**类的方法**。

```
In [10]:class Employee(object):
        def __init__(self, name, salary):
            self.name = name
            self.salary = salary

        def print_salary(self):
            print('%s: %s' % (self.name, self.salary))
```

要定义一个方法，除了第一个参数是 self 外，其他和普通函数一样。

要调用一个方法，只需要在实例变量上直接调用，除了 self 不用传递，其他参数正常传入。

```
In [11]:amy.print_salary()
```

Out[11]:

Amy Simpson: 59

通过上面的操作，我们从外部看 Employee 类，创建实例只需要给出 name 和 salary，而如何打印，都是在 Employee 类的内部定义的，这些数据和逻辑被“封装”起来了，很容易调用，但却不用知道内部实现的细节。

类的调用：

#Cl_A.py 文件：

```
class Ax:
    def __init__(self,xx,yy):
        self.x=xx
        self.y=yy
    def add(self):
        print("x 和 y 的和为： %d"%(self.x+self.y))
```

#B.py 文件：

```
from Cl_A import Ax
a=Ax(2,3)
a.add()
或
import Cl_A
a=Cl_A.Ax(2,3)
a.add()
```

以上函数和类的调用方法都是在同一个文件下的调用，对于不同文件下的调用时，需要进行说明，即需要有个“导引”，假如 Cl_A.py 文件的文件路径为：C:\Users\lenovo\Documents，现有 D:\yubg 下的 B.py 文件需要调用 Cl_A.py 文件中类 Ax 的 add 方法，调用方法如下：

```
import sys
```

```
sys.path.append(r' C:\Users\lenovo\Documents ')
```

```
import Cl_A  
a=Cl_A.Ax(2,3)  
a.add()
```

Python 在 import 函数或模块时，是在 sys.path 里按顺序查找的。sys.path 是一个列表，里面以字符串的形式存储了许多路径。使用 A.py 文件中的函数需要先将他的文件路径放到 sys.path 中。

《数据处理与分析（Python）》课程教案（8）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第四章 面向对象（第3节+第4节）					
目的、要求（分了解、理解、掌握三个层次） （1）私有属性和私有方法； （2）单例、多态、继承； （3）类属性和类方法					
重点 （1）私有属性和私有方法； （2）单例、多态、继承； （3）类属性和类方法					
难点 （1）私有属性和私有方法； （2）单例、多态、继承；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
一、属性 在 Python 中类中的属性分为两种： 公有属性和私有属性 公有属性：可以在类内类外使用，没有限制 私有属性：只能在类内使用，但可以使用方法间接访问 私有属性定义时需在前面加上两个下划线（__） <pre>class A(object): #公有属性 a = "public" #私有属性 __b = "private" print("属性为"+a) print("属性为"+__b) test = A()</pre> 二、魔法函数 类中内置方法（魔法函数） 在 Python 的类中以__XX__命名的方法都是类中的内置方法即魔法函数 如何查看类中的内置函数： <pre>print(dir(object))</pre> 三、封装 将属性和行为作为一个整体，表现生活中的事物 将属性和行为加以权限控制 在 Python 中定义类时，我们都需要将属性私有化，如何提供公开的方法去供外部访问和修改 <pre>class Animal(object): def __init__(self,name,age,sex): self.__name = name self.__age = age self.__sex = sex</pre> a = Animal("dog","2years","man") 此例子中的属性都为私有属性，那我们如何去访问？ 第一种封装方式： 在类中提供 get 方法和 set 方法供外部访问 <pre>class Animal(object): def __init__(self,name,age,sex): self.__name = name self.__age = age</pre>	

```

        self.__sex = sex

#提供公开的 get 方法
def get_name(self):
    return self.__name

#提供公开的 set 方法
def set_name(self,name):
    self.__name = name

a = Animal("dog","2years","man")
print(a.get_name())
a.set_name("cat")
print(a.get_name())

```

四、继承

类与类之间还有种关系，即继承

继承可以提高代码的利用率、复用率

object 为所有类的父类，Python 支持多继承 class Son(father1,father2.....)，在多继承中需要注意继承顺序，在继承时它会按照继承顺序来继承，即优先级。

```

class father(object):
    def __init__(self):
        self.money =100
        self.home = "房子"
    def tell(self):
        print("ok")

```

#儿子继承了父亲

```

class son(father):
    pass

```

```

if __name__ == '__main__':
    a = son()
    a.tell()
    print(a.money)
    print(a.home)

```

重写：

在子类继承父类后，父类中的方法不能满足子类的使用时，我们可以在子类中进行方法重写

方法重写时，子类与父类的方法名称、参数必须相同

在 Python 中有方法的重写，但是没有方法重载，但是可以实现方法的重载（装饰器）

方法重写示例：

```

class father(object):
    def tell(self):
        print("我喜欢平平淡淡")

```

#儿子继承了父亲

```

class son(father):
    def tell(self):
        print("我喜欢赚钱")

```

```

if __name__ == '__main__':
    a = son()
    a.tell()

```

多态基于继承的基础之上，一个对象的多种实现，即父类引用指向子类实例的现象

Python 是弱数据语言，所以多态在 python 中显得不太重要，它在定义变量的时候不用声明数据类型，因此支持多态

```
class father(object):  
    def tell(self):  
        print("我是父亲")
```

#儿子继承了父亲

```
class son(father):  
    def tell(self):  
        print("我是儿子")
```

#女儿继承了父亲

```
class daughter(father):  
    def tell(self):  
        print("我是女儿")
```

```
def test(father):  
    father.tell()  
if __name__ == '__main__':  
    a = father()  
    b = son()  
    c = daughter()  
    test(a)  
    test(b)  
    test(c)
```

阶段考核：

Python 基础语法内容

(1) 学生项目立项工作启动

《数据处理与分析（Python）》课程教案（9）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第五章 Numpy（第 1 节+第 2 节）					
目的、要求（分了解、理解、掌握三个层次） （1）numpy 对数组的处理； （2）numpy 数组的常见操作；					
重点 （1）numpy 对数组的处理； （2）numpy 数组的常见操作；					
难点 （1）numpy 对数组的处理； （2）numpy 数组的常见操作；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
【引入】 有了 list 为什么还要有 Numpy 呢？列表的元素可以是任何对象，为了保存一个简单的列表[1,2,3]，需要有 3 个指针和 3 个整数对象。对于数值运算来说，这种结构显然比较浪费内存和 CPU 计算时间。 ● 数组的创建和操作 ● 数组的计算 ● 统计基础和矩阵运算 一、数组的创建和操作 在导入 Numpy 库时，我们通过 as 将 np 作为 Numpy 的别名，导入方式如下： <pre>import numpy as np</pre> 先在 list 中创建 Numpy 数组。 <pre>In [1]: import numpy as np ...: my_list = [1, 2, 3, 4, 5] ...: my_numpy_list = np.array(my_list)</pre> <pre>In [2]: my_numpy_list Out[2]: array([1, 2, 3, 4, 5])</pre> 要想得到二维数组，则需要创建一个包含列表为元素的列表，如下所示。 <pre>In [3]: second_list = [[1,2,3], [5,4,1], [3,6,7]] ...: new_2d_arr = np.array(second_list) ...: new_2d_arr Out[3]: array([[1, 2, 3], [5, 4, 1], [3, 6, 7]])</pre> 有时为了方便数据操作，我们需要将数组转化为列表，使用 tolist()函数即可。 <pre>In [4]: c = np.array([[1, 2, 3, 4],[4, 5, 6, 7], [7, 8, 9, 10]]) ...: c Out[4]: array([[1, 2, 3, 4], [4, 5, 6, 7], [7, 8, 9, 10]])</pre> <pre>In [5]: c.tolist() Out[5]: [[1, 2, 3, 4], [4, 5, 6, 7], [7, 8, 9, 10]]</pre> <pre>In [4]: c = np.array([[1, 2, 3, 4],[4, 5, 6, 7], [7, 8, 9, 10]]) ...: c Out[5]: array([[1, 2, 3, 4], [4, 5, 6, 7], [7, 8, 9, 10]])</pre> <pre>In [6]: c.dtype #查看 c 的数据类型 Out[6]: dtype('int32')</pre> 数组的大小可以通过其 shape 属性获得。 <pre>In [7]: a.shape #查看 a 的数组维度 Out[7]: (4,)</pre> <pre>In [8]: c.shape Out[8]: (3, 4)</pre>	

数组 a 的 shape 只有一个元素 4，因此它是一维数组。而数组 c 的 shape 有两个元素，因此它是二维数组，其中第 0 轴的长度为 3，第 1 轴的长度为 4，如图所示。

还可以通过修改数组的 shape 属性，在保持数组元素个数不变的情况下，改变数组每个轴的长度。下面的例子将数组 c 的 shape 改为(4,3)，注意从(3,4)改为(4,3)并不是对数组进行转置，而是改变每个轴的大小，数组元素在内存中的位置并没有改变。

```
In [9]: c.shape = 4,3
```

```
...: c
```

```
Out[9]:
```

```
array([[ 1, 2, 3],
       [ 4, 4, 5],
       [ 6, 7, 7],
       [ 8, 9, 10]])
```

当某个轴的长度为-1 时，相当于占位符，这个-1 位置上将根据数组元素的个数自动计算此轴的长度，因此下面的代码将数组 c 的 shape 改为了(2,6)，但这里的 6 不需要人工去计算，以-1 替代，由计算机自动计算填充。

```
In [10]: c.shape = 2,-1
```

```
...: c
```

```
Out[10]:
```

```
array([[ 1, 2, 3, 4, 4, 5],
       [ 6, 7, 7, 8, 9, 10]])
```

使用数组的 reshape 方法，可以创建一个改变了尺寸的新数组，原数组的 shape 保持不变。

```
In [11]: d = a.reshape((2,2))
```

```
...: d
```

```
Out[11]:
```

```
array([[1, 2],
       [3, 4]])
```

```
In [12]: a
```

```
Out[12]: array([1, 2, 3, 4])
```

使用 reshape 方法新生成的数组和原数组共用一个内存，不管改变哪个都会互相影响。所以数组 a 和 d 其实共享数据存储内存区域，因此修改其中任意一个数组的元素都会同时修改另外一个数组的内容。

```
In [13]: a[1] = 100 # 将数组 a 的第一个元素改为 100
```

```
...: d # 注意数组 d 中的 2 也被改变了
```

```
Out[13]: array([[ 1, 100],
```

```
               [ 3, 4]])
```

数组的元素类型可以通过 dtype 属性获得。上面例子中的参数序列的元素都是整数，因此所创建的数组的元素类型也是整数，并且是 32bit 的长整型。可以通过 dtype 参数在创建时指定元素类型。

```
In [14]: np.array([[1,2,3,4],[4,5,6,7], [7,8,9,10]], dtype=np.float)
```

```
Out[14]:
```

```
array([[ 1.,  2.,  3.,  4.],
       [ 4.,  5.,  6.,  7.],
       [ 7.,  8.,  9., 10.]])
```

```
In [15]: np.array([[1,2,3,4],[4,5,6,7], [7,8,9,10]], dtype=np.complex)
```

```
Out[15]:
```

```
array([[ 1.+0.j,  2.+0.j,  3.+0.j,  4.+0.j],
       [ 4.+0.j,  5.+0.j,  6.+0.j,  7.+0.j],
       [ 7.+0.j,  8.+0.j,  9.+0.j, 10.+0.j]])
```

当想知道一个数组包含多少个数据时，可以使用 size 来查阅。

```
In [16]: d=np.array([[ 1, 100],[ 3, 4]])
```

```
...: d.size
```

```
Out[16]: 4
```

```
In [31]: len(d)
```

```
Out[31]: 2
```

注意：len 和 size 的区别，len 是指元素的个数，而 size 是指数据的个数，也就是说元素可以包含多个数据。

在本书的第 1 章关于 for 循环中学习过 range() 函数。该函数通过指定的开始值、终止值和步长生成一整数序列，但如果要生成一个小数序列呢？这就要用到 Numpy 库中 arange() 函数。arange() 函数类似于 Python 的内置 range() 函数。使用 arange() 函数需要先导入 Numpy 库。例如，产生一个 0~1 的步长为 0.1 的序列。

```
In [16]: np.arange(0,1,0.1)
```

```
Out[16]: array([ 0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])
```

linspace() 函数通过指定开始值、终止值和元素个数来创建一维数组，可以通过 endpoint 关键字指定是否包括终止值。默认设置是包括终止值。

```
In [17]: np.linspace(0, 1, 12)
```

```
Out[17]:
```

```
array([ 0. , 0.09090909, 0.18181818, 0.27272727, 0.36363636,  
0.45454545, 0.54545455, 0.63636364, 0.72727273, 0.81818182,  
0.90909091, 1. ])
```

《数据处理与分析（Python）》课程教案（10）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第五章 Pandas（180min）					
目的、要求（分了解、理解、掌握三个层次） （1）能够简要复述 Pandas 库的基本概念和功能； （2）能够编写 Pandas 中的数据结构（DataFrame、Series、Index）的增、删、改、查的代码； （3）能够实现利用 Pandas 数据导入与导出；					
重点 （1）学会 Dataframe（数据框）的使用，增、删、改、查的方法； （2）文件的导入导出；					
难点 （1）学会 Dataframe（数据框）的使用，增、删、改、查的方法； （2）文件的导入导出；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 多媒体 PPT 讲解、Jupyter notebook 举例讲解、博思智慧教学工具使用相结合。					
作业和思考题（星火生成）： 1.请简述 pandas 库的主要功能和特点。 课后作业/任务： 1. 请使用 pandas 库读取一个 CSV 文件，并显示前 5 行数据。 2. 请使用 pandas 库创建一个 DataFrame，包含以下信息：姓名、年龄、城市。然后添加一列“职业”，并将所有职业设置为“学生”。 3. 请使用 pandas 库对一个 DataFrame 进行排序，按照年龄从小到大的顺序排列。 4. 请使用 pandas 库筛选出年龄大于等于 30 岁的学生，并计算他们的平均年龄。 5. 请使用 pandas 库将一个 DataFrame 保存为 CSV 文件。					
教 学 后 记					

教 学 过 程	板 书 或 旁 注
Pandas 一、Pandas 基本介绍（星火生成，5min） Pandas 是一个开源的 Python 数据分析库，它提供了大量用于数据处理和分析的功能。Pandas 最初由 AQR Capital Management 于 2008 年 4 月开发，并于 2009 年底开源。它是基于 NumPy 的一种工具，可以非常方便地处理结构化数据。 Pandas 的核心数据结构是 DataFrame，它是一个二维表格型数据结构，可以存储多种类型的数据，如数值、字符串、布尔值等。DataFrame 具有很多实用的功能，如数据筛选、排序、分组、合并等。此外，Pandas 还提供了一些其他的数据结构，如 Series（一维数组）和 Index（索引）。 Pandas 的主要功能包括： 1. 数据导入与导出：Pandas 支持从多种文件格式导入数据，如 CSV、Excel、SQL 数据库等。同时，它也可以将数据导出为多种格式，如 CSV、Excel、HDF5 等。 2. 数据清洗：Pandas 提供了一些实用的函数，如 dropna（删除缺失值）、fillna（填充缺失值）、replace（替换值）等，可以帮助我们快速清洗数据。 3. 数据筛选与排序：Pandas 提供了丰富的筛选和排序功能，如 loc（基于标签的筛选）、iloc（基于位置的筛选）、sort_values（排序）等。 4. 数据分组与聚合：Pandas 支持对数据进行分组和聚合操作，如 groupby（分组）、agg（聚合）等。这些功能可以帮助我们对数据进行更深入的分析。 5. 数据合并与连接：Pandas 提供了一些实用的函数，如 concat（纵向连接）、merge（横向连接）等，可以帮助我们合并多个数据集。 6. 时间序列分析：Pandas 内置了对时间序列数据的支持，如 Timestamp（时间戳）、Timedelta（时间差）等。此外，它还提供了一些专门用于时间序列分析的函数，如 resample（重采样）、rolling（滚动窗口计算）等。 7. 可视化：虽然 Pandas 本身不提供可视化功能，但它可以与 Matplotlib、Seaborn 等可视化库无缝集成，帮助我们快速生成各种图表。 总之，Pandas 是一个非常强大的 Python 数据分析库，它可以帮助我们轻松地处理和分析各种类型的结构化数据。学习 Pandas 对于从事数据分析工作的人来说是非常有帮助的。 Pandas 是数据处理和数据分析首选库。Pandas 库提供的数据结构非常灵活，富有表现力，其数据结构经过特殊的设计，使得数据分析更加容易。使用 Pandas 库时，需要导入它并命名别名为 pd： <pre>import pandas as pd</pre> 二、序列 series（10min） Series：序列（也称系列）用于存储一行或一列的数据，以及与之相关的索引的集合，使用方法如下。 <pre>Series([数据 1, 数据 2,...], index=[索引 1, 索引 2,...])</pre>	Pandas 基本信 息 用 PPT 进 行 讲 解 呈 现； PPT 简 单 讲 解

```

from pandas import Series
X = Series(['a',2,'中国'],index=['first','second','third']) #自定义索引
X

X = Series(['a',2,'中国'])#忽略索引，默认从 0 开始 X

X[2] #访问序列的某个值，类似于切片操作

se = Series([1,2,3])
X.append(se) #给序列新增值，只能是添加序列

x = Series(['a', True, 1], index=['first', 'second', 'third'])
x

x.drop('first') #按索引名删除

```

配合
jupyter
notebook
进行代
码演示
与结果
互动

三、DataFrame (70min)

DataFrame 是用于存储多行和多列的数据集合，类似于 Excel 的二维表格。对于 DataFrame 的操作较多的是“增、删、改、查”。

注意：DataFrame 单词是驼峰写法，索引不指定时也可以省略。使用数据框时，要先从 pandas 中导入 DataFrame 包，数据框中的数据访问方式如下表。

	age	name	← 列名
0	26	Ken	
1	29	Jerry	
2	24	Ben	

↑ 索引 ↑ 位置访问: df.at[2, 'name']

PPT 简
单讲解

访问位
置

方法

备注

访问列	变量名[列名]	访问对应的列，如 df['name']
访问行	变量名[n: m]	访问 n 行到 m-1 行的数据，如 df[2:3]
访问块 (行和列)	变量名.iloc[n1:n2,m1:m2]	访问 n1 到 (n2-1) 行、m1 到 (m2-1) 列的数据，如 df.iloc[0:3,0:2]
访问位置	变量名.at[行名,列名]	访问(行名,列名)位置的数据，如 df.at[1,'name']或者 df.loc[2,'name']

创建数据框（该部分代码由星火生成后进行修改）

```
from pandas import DataFrame

dic = {'name':['Jerry','Tom','Jane'],'tel':[6601,6602,6603]}
df1 = DataFrame(dic,index=['a','b','c']) #以字典的形式创建数据框
df1

import numpy as np
arr = np.random.randint(10, 15, size=(2,6)) #产生一个（2，6）的 12 个数的数组
ar=arr.reshape(6,2)
ar

df2 = DataFrame(ar) #以数组的形式创建数据框
df2

#提取列名
df2.columns

df2.columns = ['a','b'] #修改列名
df2

df2.rename(columns={'a':'A'},inplace=True) #利用字典的方式修改某一列或者几列，
inplace=True 表示在原数据上修改，否则修改完了重新生成新的数据
df2

# 根据索引重新构建数据
df3 = df2.reindex([1,2,5]) # 索引存在就正常显示数据，不存在的索引其数据就是 NaN
df3
```

配合
jupyter
notebook
进行代
码演示
与结果
互动

通过 `reindex` 的功能，我们可以理解为用此法可以提取某些行的数据

```
df3.index=['a','b','c'] #修改索引号
```

```
df3
```

```
df1.reset_index(drop=True, inplace=True) #重新到默认的 0 开始索引
```

```
df1
```

访问数值（查）（该部分代码由星火生成后进行修改）

- 1、访问某一列
- 2、访问某一行
- 3、访问某一个值（某一行列交叉位置）
- 4、访问某一个块（）多行多列交叉位置

```
df2['A'] #访问某列
```

```
#访问多列
```

```
df2[['A','b']]
```

```
df3
```

```
df3.loc['b'] #按索引名访问某行
```

```
df1.iloc[2] #按索引号访问某行
```

```
df1.loc[2] #当没有索引名时，同索引号
```

```
#访问 6602
```

```
df1.at[1,'tel'] #访问某个值
```

```
df1.iloc[0:2,0:2] #访问某个块
```

```
#查看数据的前 n 行后 n 行
```

```
df2.head(2) #默认是前 5 行
```

```
df2.tail()
```

增加行列

增加行与排序

```
import numpy as np
import pandas as pd
from pandas import DataFrame
```

#创建一个数据框

```
arr = np.arange(9)
ar = arr.reshape(3,3)
df = DataFrame(ar)
df
```

【插入数据行】

```
df.loc[5]=['a','b','c']    #指定的位置插入一行，行的索引名必须是不存在的，且长度同
现有的行数据长度
df
```

注意：插入的行索引名必须是不存在的，且长度同现有的行数据长度。

```
df0 = pd.DataFrame([6,6,6]).T #创建一个新数据框
df0
```

合并数据框

```
dic = {0:9,1:10,2:12}
df.append(dic,ignore_index=True)    #以字典的方式插入一行
```

```
pd.concat([df,df0],ignore_index=True) #将 df0 合并到 df 中并生成新的数据框，
ignore_index=True 表示新的数据框的索引被默认改变顺延
```

增加列与排序（该部分代码注释由星火生成）

增加一列很简单，类似于字典，若存在则覆盖，没有则新增一列，前提是该列与已有的列长度相等。

```
df['new'] = ['a','b','c','d']#插入列
df
```

```
df.insert(2,'column_c',[4,0,2,1])    #在指定的位置插入一列
df
```

【按列排序，并重置索引】

```
df1 = df.sort_values(by= 'column_b')#按指定的列排序
df1
```

学 生 练
习，加深
印象

```
df1.reset_index()    #重置索引
```

删除行列（该部分代码注释由星火生成）

删除列

```
df
```

```
#### 【删除列】
```

```
df.drop('column_b',axis=1,inplace=True) #删除列，inplace=True 表示在原数据上  
df
```

```
df.drop(df.columns[[0,1]],axis=1,inplace=False)  #删除多列
```

删除行

```
df
```

```
df.drop(1,axis=0,inplace=False)  #按照索引号/名删除行时 axis=0 可省略，inplace =  
False 时也可以省略
```

```
df
```

```
df.drop(df.index[3],inplace=True) #按照索引号删除  
df
```

```
df.drop(df.index[[1,2]],inplace=False) #删除多个行
```

删除特定数值的行,其实是筛选

```
df
```

```
~df['new'].str.contains('c')
```

```
df[~df['new'].str.contains('c')]  #删除列名为 2 的包含字母 c 的行
```

```
df[df[2]>4]  #删除列名为 1 的数值中小于等于 4 的行
```

学 生 练
习，加深
印象

改

将能访问到的数据直接赋值覆盖即可以改

查

在某列或者全局中查找某个具体的值

```
d = {'a':[1,2,3,2], 'b':[0,2,1,2], 'c':[0,2,1,2]}
df = DataFrame(d)
df
```

```
#查找数据 3
df[df['a']==3]
```

```
#也可以用 where 处理
np.where(df==3) #查找制定的元素或者值
```

查找空值

```
#创建一个含空值的数据框
d = {'a':[1,2,3,2], 'b':[0,2,1,2], 'c':[0,2,np.NAN,2]}
df = DataFrame(d)
df

np.where(np.isnan(df)) #但是查找空值 Nan 时最好用 np.isnan(df)
```

【注】但是查找空值 Nan 时最好用 np.isnan(df),用 df == np.nan 容易出错。

定位重复数据

```
df

df_0 = df.duplicated('a') #显示各行是否重复， bool 类型
df_0
```

删除重复数据

学 生 练
习，加深
印象

使用 drop_duplicates 方法

```
DataFrame.drop_duplicates(subset=None, keep='first', inplace=False)
```

subset 参数：根据哪个字段进行重复筛选（多个字段就写成列表形式） inplace

参数：是否在原数据集更改 keep 参数：是从头开始筛选还是从末尾数据开始

筛选（{'first', 'last', False}, default 'first'）

```
df
```

```
df.drop_duplicates() #删除重复行
```

```
df
```

#也可以直接用删除行操作

```
df.drop(3, axis=0, inplace = False)
```

数据准备(数据清洗)

数据的导入(该部分代码由星火生成后进行修改)

对于常用的数据我们有 csv、txt、xlsx 等格式，当然还有从 mysql 数据导入等。我们这里对 csv、txt、xlsx 三种格式进行讲解操作。

1. 导入 txt 文件数据

导入格式如下：

```
read_table(file, names=[列名 1, 列名 2, ..], sep="...",...)
```

file 文件路径与文件名

names 列名，默认为文件中的第一行作为列名

sep 分隔符，默认为空

```
import pandas as pd
```

```
df_txt = pd.read_table(r'news_data.txt')
```

```
df_txt
```

2. 导入 csv 文件数据

导入格式如下：


```
read_csv(file,names=[列名 1, 列名 2, ..],sep="...",...)
file      文件路径与文件名
names     列名，默认为文件中的第一行作为列名
sep       分隔符，默认为空，表示默认导入为一列
```

```
df_csv = pd.read_csv(r'data.csv',encoding = 'gbk')
df_csv
```

读取 csv 格式的文件也可使用 `read_table()`，结果与 `read_csv()`一致。

3. 导入 excel 文件数据

Excel 文件包含.xlsx 和.xls 两种格式，导入格式如下：

```
read_excel(file, sheet_name,header=0)
file      文件路径与文件名
sheet_name sheet 的名称，如：sheet1
header    列名，默认为 0(只接受布尔型 0 和 1)，文件的第一行作为列名
```

注：在版本较低的 pandas 中(大概是低于 0.21 版本)，`sheet_name` 参数需改为 `sheetname`。

查阅 Pandas 的版本号代码：`print(pd.__version__)`。

```
from pandas import DataFrame
from pandas import read_excel
df_excel = read_excel(r'i_nuc.xls',sheet_name='Sheet4')
df_excel.head()      #展示数据表的前 5 行，显示后 5 行为 df.tail()
```

数据的导出(该部分代码由星火生成后进行修改)

1. 导出数据为 csv 文件：

```
to_csv(file_path,sep= ", ", index=TRUE, header=TRUE)
file_path 文件路径
sep        分隔符，默认是逗号
index      是否导出行序号，默认是 TRUE，导出行序号
header     是否导出列名，默认 TRUE 导出列名
```

2. 导出 excel 文件：

```
to_excel(file_path, index=TRUE,header=TRUE)
```

file_path 文件路径

index 是否导出行序号，默认是 TRUE，导出行序号

header 是否导出列名，默认 TRUE 导出列名

```
df = df_excel.copy()
```

```
df
```

```
df.to_csv(r'csv_1.csv',sep=',') #保存在桌面
```

```
df.to_excel(r'excel_1.xlsx')
```

由于编码问题，在导出数据后，直接打开数据会显示乱码。为了解决这个问题，在导出数据时需要进行 **encoding** 编码。

csv、excel 一般编码为 utf_8_sig 格式。如：

```
import pandas as pd
```

```
path = r"data.csv"
```

```
df = pd.read_csv(path)
```

```
print(df)
```

```
filename = r"c:\Users\yubg\Desktop\restuarant.csv"
```

```
df.to_csv(filename, encoding='utf_8_sig')
```

四、总结与学生练习（5min）

进行简要总结，布置课后任务。学生完成数据的导入导出练习。

《数据处理与分析（Python）》课程教案（11）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第六章 数据处理					
目的、要求（分了解、理解、掌握三个层次） (1) 处理缺失数据以及清除无意义的信息； (2) 数据的抽取方法，字段的拆分、记录抽取、随机抽样、重新索引等； (3) 数据合并方法，concat、字段合并、字段匹配等；					
重点 (1) 处理缺失数据以及清除无意义的信息； (2) 数据的抽取方法，字段的拆分、记录抽取、随机抽样、重新索引等； (3) 数据合并方法，concat、字段合并、字段匹配等；					
难点 (1) 处理缺失数据以及清除无意义的信息； (2) 数据的抽取方法，字段的拆分、记录抽取、随机抽样、重新索引等； (3) 数据合并方法，concat、字段合并、字段匹配等；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
一、pandas 合并数据方法 pandas 中合并数据大概有 5 个函数： • join 主要用于基于索引的横向合并拼接； • merge 主要用于基于指定列的横向合并拼接； • concat 可用于横向和纵向合并拼接； • append 主要用于纵向追加； • combine 可以通过使用函数，把两个 DataFrame 按列进行组合。 <pre>In []: import pandas as pd</pre> join join是基于索引的横向拼接，如果索引一致，直接横向拼接。如果索引不一致，则会用Nan值填充。 <pre>In []: x = pd.DataFrame({'A': ['A0', 'A1', 'A2'], 'B': ['B0', 'B1', 'B2']}, index=[0, 1, 2]) y = pd.DataFrame({'C': ['C0', 'C2', 'C3'], 'D': ['D0', 'D2', 'D3']}, index=[0, 1, 2]) print(x, "\n\n", y)</pre> <pre>In []: x.join(y)</pre> merge merge是在多个数据表中，基于某一个键（列）来匹配合并多个数据表。该函数的典型应用场景是，针对同一个主键存在两张不同字段的表，根据主键整合所有的字段到一张表里面。可以指定不同的how参数，表示连接方式，有inner内连、left左连、right右连、outer全连，默认为inner;当然可以指定按那个字段进行匹配，用参数on。 <pre>In []: x = pd.DataFrame({'姓名': ['张三', '李四', '王五'], '学号': ['123', '120', '124'], '班级': ['一班', '二班', '三班']}) y = pd.DataFrame({'专业': ['统计学', '计算机', '绘画'], '学号': ['123', '124', '121'], '班级': ['一班', '三班', '四班']}) print(x, "\n\n", y)</pre> <pre>In []: pd.merge(x,y,how="left")</pre> <pre>In []: pd.merge(x,y,on = '学号',how="left")</pre> concat concat函数既可以用于横向拼接，也可以用于纵向拼接。 <pre>In []: x = pd.DataFrame([['Jack','M',40],['Tony','M',20]], columns=['name','gender','age']) y = pd.DataFrame([['Mary','F',30],['Bob','M',25]], columns=['name','gender','age']) print(x, "\n\n", y)</pre> <pre>In []: #纵向拼接 z = pd.concat([x,y],axis=0) z</pre> <pre>In []: #横向拼接 z = pd.concat([x,y],axis=1) z</pre>	

combine

combine可以通过使用函数，把两个DataFrame按列进行组合。

```
In [ ]: import numpy as np
x = pd.DataFrame({"A": [3,4], "B": [1,4]})
y = pd.DataFrame({"A": [1,2], "B": [5,6]})
print(x, "\n\n", y)
```

```
In [ ]: x.combine(y, lambda a, b: np.where(a > b, a, b))
```

注：combine函数，用于返回对应位置上的最大值。

markdown创建表格

表格字段一	表格字段二	表格字段三	表格字段四
默认居中对齐	左对齐	右对齐	居中对齐
cell	cell	cell	cell

append

append主要用于纵向追加数据。

```
In [ ]: x = pd.DataFrame([['Jack', 'M', 40], ['Tony', 'M', 20]], columns=['name', 'gender', 'age'])
y = pd.DataFrame([['Mary', 'F', 30], ['Bob', 'M', 25]], columns=['name', 'gender', 'age'])
print(x, "\n\n", y)
```

```
In [ ]: x.append(y)
```

二、处理缺失数据

检测与处理重复值

```
import pandas as pd
```

```
detail_duplicates = pd.read_csv('data/detail_duplicates.csv')
detail_duplicates
```

	order_id	dishes_name	counts	amounts
0	NaN	蒜蓉生蚝	1.0	49.0
1	NaN	NaN	1.0	48.0
2	417.0	大蒜苋菜	1.0	30.0
3	417.0	芝麻烤紫菜	1.0	25.0
4	417.0	蒜香包	1.0	13.0
5	NaN	NaN	NaN	NaN
6	301.0	香烤牛排	1.0	55.0
7	417.0	芝麻烤紫菜	1.0	25.0
8	301.0	芝麻烤紫菜	1.0	25.0
9	301.0	番茄有机花菜	1.0	32.0
10	417.0	蒙古烤羊腿	1.0	48.0

```
[11]: detail_duplicates.drop_duplicates()
detail_duplicates.drop_duplicates(subset=['order_id'], keep='last')
detail_duplicates.drop_duplicates(subset=['order_id', 'dishes_name'], keep='last')
```

检测与处理缺失值

```
print(detail_duplicates)
detail_duplicates.isnull()
detail_duplicates.notnull()

detail_duplicates.dropna(axis=0, how='any') # 删除行数据
detail_duplicates.dropna(axis=1, how='all') # 删除列数据
detail_duplicates.dropna(axis=0, how='any', subset=['dishes_name', 'amounts']) # 删除行数据

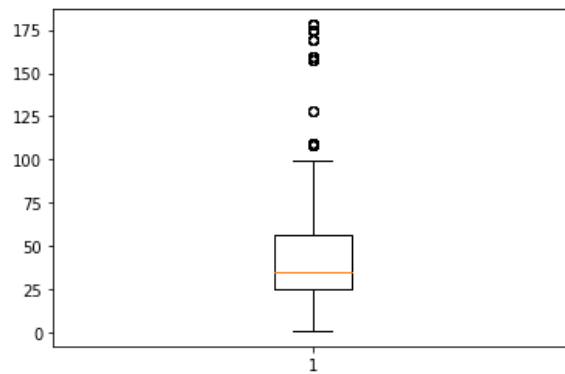
detail_duplicates.fillna(1) # 使用固定值替换缺失值
detail_duplicates.fillna(method='bfill') # 使用后面元素替换缺失值
```

检测与处理异常值

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
```

```
[2]: data = pd.read_excel('data/meal_order_detail.xlsx')
data.head()
```

```
15]: plt.boxplot(data['amounts'])  
plt.show()
```



```
16]: # 自定义函数用于将数据中的异常值替换为缺失值  
def replace(x):  
    import numpy as np  
    QU = x.quantile(0.75)  
    QL = x.quantile(0.25)  
    IQR = QU - QL  
    x[(x > (QU + 1.5*IQR)) | (x < (QL - 1.5*IQR))] = np.nan  
    return x
```

```
17]: data['amounts'].isnull().sum()
```

```
17]: 0
```

```
18]: replace(data['amounts']).isnull().sum()
```

《数据处理与分析（Python）》课程教案（12）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第六章 数据处理					
目的、要求（分了解、理解、掌握三个层次） （1）数据计算，通过对各字段的进行加、减、乘、除等四则算术运算，计算出来新的字段； （2）数据标准化； （3）数据分组，会使用 cut，新增一列，将原来的数据按照其性质归入新的类别中；					
重点 （1）数据计算，通过对各字段的进行加、减、乘、除等四则算术运算，计算出来新的字段； （2）数据标准化； （3）数据分组，会使用 cut，新增一列，将原来的数据按照其性质归入新的类别中；					
难点 （1）数据计算，通过对各字段的进行加、减、乘、除等四则算术运算，计算出来新的字段； （2）数据标准化； （3）数据分组，会使用 cut，新增一列，将原来的数据按照其性质归入新的类别中；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
数据标准化 常用的归一化方法 数据标准化（归一化）：是数据分析和挖掘的一项基础工作，不同评价指标往往具有不同的量纲和量纲单位，这样的情况会影响到数据分析的结果，为了消除指标之间的量纲影响，需要进行数据标准化处理，以解决数据指标之间的可比性。 原始数据经过数据标准化处理后，各指标处于同一数量级，适合进行综合对比评价。 常用的归一化方法： 一、min-max 标准化（Min-Max Normalization） 也称为离差标准化，是对原始数据的线性变换，使结果值映射到[0 – 1]之间。转换函数如下： $x^*=(x-\min)/(\max-\min)$ 其中 max 为样本数据的最大值，min 为样本数据的最小值。 这种方法有个缺陷就是当有新数据加入时，可能导致 max 和 min 的变化，需要重新定义。 <pre>In []: #导入并查看数据 from pandas import read_excel df = read_excel(r'data_df.xlsx',sheetname='Sheet1')#采用上次实验课上已经处理好的数据data_df.xlsx df In []: #查看数据，发现军训数据列的分数比较高 print(df.军训.max(),df.军训.min()) In []: #查看所有的课程列的最高分和均值 #先提取列名 col = df.columns.tolist() col In []: for i in col[5:]: print(i,df[i].max(),df[i].mean()) In []: #对数分列进行数据标准化并展示数据 scale= (df.数分.astype(int)-df.数分.astype(int).min())/(df.数分.astype(int).max()-df.数分.astype(int).min()) scale.head() In []: #对所有的课程列进行数据标准化 for i in col[5:]: df[i]= (df[i].astype(int)-df[i].astype(int).min())/(df[i].astype(int).max()-df[i].astype(int).min()) df In []: df_1 = df.drop('Unnamed: 0',axis=1) #删除导入的数据中序号列 In []: df_1.to_excel('df_1.xlsx') #将标准化的数据保存备用 In []: #也可以使用MinMaxScaler(最小最大值标准化,0-1标准化) 函数来标准化数据，省去了自己写代码 import pandas as pd from sklearn import preprocessing df0 = pd.DataFrame({'a':[1,2,3,4,5,6,7,8,9,10], 'b':[1,2,3,4,5,6,7,8,9,10]}) df0.columns=['数分','解几'] df0['解几']=df['解几'] df0['数分']=df['数分'] df0.shape In []: min_max_scaler = preprocessing.MinMaxScaler() X_train_minmax = min_max_scaler.fit_transform(df0) X_train_minmax</pre> 注意：结论和上面的一致。只是这个MinMaxScaler函数仅适用于2维及其以上的数据。 二、Z-score 标准化方法 z-score 标准化方法适用于属性 A 的最大值和最小值未知的情况，或有超出取值范围的离群数据的情况。	

这种方法给予原始数据的均值（mean）和标准差（standard deviation）进行数据的标准化。经过处理的数据符合标准正态分布，即均值为 0，标准差为 1，转化函数为：

$$x^* = (x - \mu) / \sigma$$

其中 μ 为所有样本数据的均值， σ 为所有样本数据的标准差。

使用 sklearn.preprocessing.scale() 函数，可以直接将给定数据进行标准化：

```
In [ ]: from sklearn import preprocessing
import numpy as np

df1=df['数分']

df_scaled = preprocessing.scale(df1)
df_scaled
```

```
In [ ]: df_scaled.mean(axis=0)    #均值
```

```
In [ ]: df_scaled.std(axis=0)    #标准差
```

也可以使用sklearn.preprocessing.StandardScaler类
使用该类的好处在于可以保存训练集中的参数（均值、方差）直接使用其对象转换测试数据集：

```
In [ ]: #例如:
X = np.array([[ 1., -1.,  2.],[ 2.,  0.,  0.],[ 0.,  1., -1.]]) #创建数组
```

```
In [ ]: scaler = preprocessing.StandardScaler().fit(X)    #数据标准化并训练数据集
scaler.mean_    #均值
```

```
In [ ]: scaler.scale_    #标准差
```

```
In [ ]: scaler.transform(X)    #转化标准化数据
```

```
In [ ]: #可以直接使用训练集对测试集数据进行转换
scaler.transform([[ -1.,  1.,  0.]])
```

三 Z-scores 简单化

模型如下：

$$x^* = 1 / (1 + x)$$

x 越大证明 x* 越小，这样就可以把很大的数规范在[0-1]之间了。

四、转换数据

```
import pandas as pd
data = pd.read_excel('data/meal_order_detail.xlsx')
data.head(3)

type(data['amounts'].iloc[1])
```

哑变量处理

```
pd.get_dummies(data['dishes_name'])#

data['dishes_name'].value_counts()
```

离散化连续型数据

```
pd.cut(data['amounts'], 5)
```

四、总结

- 一、min-max 标准化: $x*=(x-\min)/(\max-\min)$
 - 二、Z-score 标准化方法: $x*=(x-\mu)/\sigma$
 - 三、Z-scores 简单化: $x*=1/(1+x)$
- 总结：以上一、二方法都需要依赖样本所有数据，而 3 方法只依赖当前数据，可以动态使用。

《数据处理与分析（Python）》课程教案（13）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第六章 数据处理					
目的、要求（分了解、理解、掌握三个层次） (1) 学生能够理解并掌握 DataFrame 的基本操作，如创建、索引、切片等。 (2) 学生能够使用 numpy 计算 DataFrame 的均值、标准差等统计量。 (3) 学生能够处理时间序列数据，包括转换时间格式、提取时间信息、时间平移等。 (4) 学生能够使用分组聚合进行组内计算，包括对不同列执行不同操作。					
重点 (1) 分组分析方法，使用 groupby，以及常用的统计指标：计数、求和、平均值；					
难点 (1) 分组分析方法，使用 groupby，以及常用的统计指标：计数、求和、平均值；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
内容概览： 1. DataFrame 基本操作 2. numpy 计算 DataFrame 统计量 3. 时间序列数据处理 4. 分组聚合进行组内计算 教学内容： 一、描述分析 DataFrame 数据 <pre>import numpy as np d=[[1.3,2.0,3,4],[2,4,1,4],[2,5,1.9,7],[3,1,0,11]] df = pd.DataFrame(d, index=['a', 'b', 'c', 'd'], columns=['A', 'B', 'C', 'D']) print(df) print(np.mean(df, axis=1)) df.mean(axis=1) df.std() df.describe() df['A'].value_counts()</pre> 二、转换与处理时间序列数据 <pre>import pandas as pd order = pd.read_csv('data/meal_order_info.csv', encoding='gbk') # print(order) print(order['lock_time'].dtypes) order['lock_time'] = pd.to_datetime(order['lock_time']) print(order['lock_time'].dtypes) print(pd.DatetimeIndex(order['lock_time'])) print(pd.PeriodIndex(order['lock_time'], freq='H')) order['lock_time'] print(order['lock_time'][0].year) # 获取数据年份信息 print(order['lock_time'].dt.year) # 获取数据年份信息</pre>	实践：让学生动手操作，解决实际问题。

```
print(order['lock_time'].dt.month)    # 获取数据月份信息
print(order['lock_time'].dt.week)     # 获取数据周次信息

print(order['lock_time'] + pd.Timedelta(days=1))    # 时间平移
print(order['lock_time'][1] - order['lock_time'][0]) # 求时间差别
```

三、使用分组聚合进行组内计算

```
import pandas as pd

detail = pd.read_excel('data/meal_order_detail.xlsx')
detail.head()

detail_group = detail[['order_id', 'counts',
'amounts']].groupby(by='order_id')    # 分组操作

detail_group.agg('mean').head(3)    # 对分组数据的所有列都执行 mean 操作

detail_group.agg(['mean', 'sum']).head(3)    # 对分组数据的所有列都执行
mean 和 sum 操作

detail_group.agg({'counts': ['mean', np.max], 'amounts': 'std'}).head(3)    # 对分
组数据的不同列执行不同操作

detail_group.agg({'counts': lambda x: sum(x)**2}).head(3)    # 将自定义函数
放入聚合操作中
```

总结：回顾本节课的主要内容，强调重点和难点。

《数据处理与分析（Python）》课程教案（14）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第六章 数据处理					
目的、要求（分了解、理解、掌握三个层次） （1）交叉分析方法，掌握 pivot_table 的使用； （2）结构分析方法，在分组的基础上，计算各组成部分所占的比重，进而分析总体的内部特征； （3）相关分析方法，研究现象之间是否存在某种依存关系，并对具体有依存关系的现象探讨其相关方向以及相关程度；					
重点 （1）交叉分析方法，掌握 pivot_table 的使用；					
难点 （1）交叉分析方法，掌握 pivot_table 的使用；					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
一、创建透视表与交叉表 <pre>import pandas as pd detail = pd.read_excel('data/meal_order_detail.xlsx') detail.head() pd.pivot_table(detail[['order_id', 'counts', 'amounts']], index='order_id', aggfunc='sum').head(3) pd.pivot_table(detail[['order_id', 'dishes_name', 'counts']], index='order_id', columns='dishes_name',aggfunc='sum').head(3) pd.pivot_table(detail[['order_id', 'dishes_name', 'counts']], index='order_id', columns='dishes_name',values='counts', fill_value=0).head() pd.crosstab(index=detail['order_id'], columns=detail['dishes_name']).head(3) pd.crosstab(index=detail['order_id'], columns=detail['dishes_name'], values=detail['counts'], aggfunc='sum').fillna(0).head(3)</pre> 二、教学示例代码： <pre>import pandas as pd # 创建一个示例数据集 data = {'A': ['foo', 'bar', 'baz', 'foo', 'bar', 'baz'], 'B': ['one', 'two', 'three', 'two', 'three', 'one'], 'C': [1, 2, 3, 4, 5, 6], 'D': [10, 20, 30, 40, 50, 60]} df = pd.DataFrame(data) # 使用 pivot_table 进行交叉分析 pivot_table = df.pivot_table(values='D', index='A', columns='B', aggfunc=np.sum) print(pivot_table) # 计算各组成部分所占的比重 total = df['C'].sum() proportions = df.groupby('A')['C'].sum() / total print(proportions)</pre>	学生跟练

研究现象之间是否存在某种依存关系

```
correlation = df[['C', 'D']].corr()
```

```
print(correlation)
```

《数据处理与分析（Python）》课程教案（15）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第 8 章 数据可视化					
目的、要求（分了解、理解、掌握三个层次） （1） 学生能够理解数据可视化的概念和重要性。 （2） 学生能够掌握画饼图、散点图、折线图、柱形图和直方图的基本方法和技巧。 （3） 学生能够运用所学知识进行实际数据的可视化分析。					
重点 学生能够掌握画饼图、散点图、折线图、柱形图和直方图的基本方法和技巧。					
难点 学生能够掌握画饼图、散点图、折线图、柱形图和直方图的基本方法和技巧。					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板书或旁注
数据的可视化 matplotlib 数据可视化是关于图形或表格的数据展示。旨在借助于图形化手段，清晰有效地传达和沟通信息。有研究表明，人类大脑接收或理解图片的速度要比文字快 6 万倍，所以再整齐的数据，再好的表格，也不抵一张图来的简单、快捷。先画一个 $y=x*x$ 的图像。 #加载模块 <pre>import matplotlib.pyplot as plt %matplotlib inline %config InlineBackend.figure_format = "retina" # 在屏幕上显示高清图片</pre> 如果在 Jupyter Notebook 中，为了方便图形的显示，需要加入显示图像方式的代码： <pre>%matplotlib inline %config InlineBackend.figure_format = "retina"</pre> 这个命令在绘图时，将图片内嵌在交互窗口，而不是弹出一个图片窗口，但是有一个缺陷：除非将代码一次执行，否则无法叠加绘图。在分辨率较高的屏幕（例如 Retina 显示屏）上，Jupyter Notebook 中的默认图像可能会显得模糊，可以在 <code>%matplotlib inline</code> 之后使用 <code>%config InlineBackend.figure_format = 'retina'</code> 来呈现分辨率较高的图像。 <pre>import numpy as np x = np.arange(-10,11) y1 = x**2 y2 = x+50 plt.plot(x, y1) plt.plot(x, y2) #plt.axis("equal")</pre>	

线图

点图和线图可以用来表示二维数据之间的关系，是查看两个变量之间关系的最直观的方法。可以通过 `plot()` 来得到。

```
import numpy as np

x = np.arange(-10,11)
y1 = x**2

y2 = x+50

plt.plot(x, y1)
plt.plot(x, y2)
```

点图

点图 `scatter()` 函数。

```
scatter(x,y,c='r',s=value,linewidths=lValue,marker='o')
```

参数说明：

- x: 数组
- y: 数组
- c: 表示颜色
- s: 表示点的大小
- linewidths: 点的形状边线条粗细
- marker: 点的形状

其中颜色 b 表示 blue，c 表示 cyan，g 表示 green，k 表示 black，r 表示 red，w 表示 white，y 表示 yellow。

形状的表示有，‘.’表示点，‘o’表示圆圈，‘D’表示钻石，‘*’表示五角星。

大小 s 默认为 20，s=0 时点不显示；颜色 c 默认为蓝色 b。 <code> # 简单的例子

```
import matplotlib.pyplot as plt

import numpy as np

x = list(range(1, 7))

plt.scatter(x, x, s=10*np.array(x)**2, c=x)
```

```
plt.show()
```

饼图

```
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']

# 构造数据
edu = [0.2515,0.3724,0.3336,0.0368,0.0057]
labels = ['中专','大专','本科','硕士','其他']

# 绘制饼图
plt.pie(x = edu,labels=labels) #可显示占比数据，添加参数：autopct='%1f%%'
plt.show()
```

柱状图

绘制柱状图用 `bar()` 函数。

`bar()` 的构造函数: `bar(x, height, width, *, align='center', **kwargs)`

`x`: 包含所有柱子的下标的列表 `height`: 包含所有柱子的高度值的列表 `width`: 每个柱子的宽度。可以指定一个固定值，那么所有的柱子都是一样的宽。或者设置一个列表，这样可以分别对每个柱子设定不同的宽度。 `align`: 柱子对齐方式，有两个可选值: `center` 和 `edge`。`center` 表示每根柱子是根据下标来对齐, `edge` 则表示每根柱子全部以下标为起点，然后显示到下标的右边。如果不指定该参数，默认值是 `center`。

```
import matplotlib.pyplot as plt
import numpy as np

# 创建一个点数为 8 x 6 的窗口，并设置分辨率为 80 像素/每英寸
plt.figure(figsize=(8, 6), dpi=80)

#初始值
N = 6 # 柱子总数
index = np.arange(N) # 包含每个柱子下标的序列
width = 0.35 # 柱子的宽度

# 包含每个柱子对应值的序列
values = (25, 32, 34, 20, 41, 50)
```

```

# 绘制柱状图，每根柱子的颜色为紫罗兰色
p2 = plt.bar(index, values, width, label="rainfall", color="#87CEFA")

plt.xlabel('Months') # 设置横轴标签
plt.ylabel('rainfall (mm)') # 设置纵轴标签
plt.title('Monthly average rainfall') # 添加标题

plt.xticks(index, ('Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'), # 添加纵横轴的刻度
            rotation=40, color="r", size=25) # 将 x 轴上标度倾斜 40°, 红色,
# 字体大小为 25
plt.yticks(np.arange(0, 81, 10))

# 添加图例
plt.legend(loc="upper right")

# 将柱顶标注数值
for a, b in zip(index, values):
    plt.text(a, b, '%.0f' % b, ha='center', va='bottom', fontsize=11)

plt.show()

```

直方图

hist 函数调用方式:

```

n, bins, patches = plt.hist(arr,
                             bins=10,
                             density=0,
                             facecolor='black',
                             edgecolor='black',
                             alpha=1,
                             histtype='bar')

```

hist 的参数非常多，但常用的就这几个，只有第一个是必须的，其他的可选

arr: 直方图的一维数组 x

bins: 直方图的柱数，可选项，默认为 10

density: 是否将得到的直方图向量归一化，默认为 0

facecolor: 直方图颜色

edgecolor: 直方图边框颜色

alpha: 透明度

histtype: 直方图类型，'bar', 'barstacked', 'step', 'stepfilled'

返回值 :

n: 返回每段里包含的数据
bins: 返回各个 bin 的区间范围
patches: 返回每个 bin 里面包含的数据，是一个 list

```
import numpy as np
import matplotlib.mlab as mlab
import matplotlib.pyplot as plt
data = np.random.randint(1,100,50)
data

n,bins,patches=plt.hist(data,10,color='steelblue')
plt.show()
print(n,bins,list(patches)[0])
```

pyecharts

ECharts，缩写来自商业级数据图表 Enterprise Charts，Echarts 是百度开源的一个数据可视化纯 Javascript(JS) 库。ECharts 主要用于数据可视化，ECharts 支持图表类型有柱状图（条状图）、散点图（气泡图）、饼图（环形图）、地图、折线图（区域图）、雷达图（填充雷达图）、K 线图、和弦图、力导向布局图、仪表盘、漏斗图、事件河流图等 12 类图表，同时提供标题，详情气泡、图例、值域、数据区域、时间轴、工具箱等 7 个可交互组件，支持多图表、组件的联动和混搭展现。

在下载安装之前，我们必须对 pyecharts 的版本了解一下。pyecharts 分为 v0.5.X 和 v1 两个大版本，相互不兼容，v1 是一个全新的版本。V0.5.X 支持 Python2.7，3.4+，经开发团队决定，0.5.x 版本将不再进行维护。V1 仅支持 Python3.6+，新版本系列将从 v1.0.0 开始。

学习 Python 最怕的就是版本不兼容性问题，为了读者适应最新版，此处我们选择 Python3.7 和 V1。至于它有什么新的动向，可以查阅网站：<https://pyecharts.org/#/zh-cn/quickstart>。

伪代码：

```
pie = Pie()
pie.add("",data)          # bar 图则为 add_xaxis()和 add_yaxis()
#pie.set_global_opts(title_opts=opts.TitleOpts(title="主标题", subtitle="副标题")) #或者直接使用字典参数，如下：
```

```
pie.set_global_opts(title_opts={"text": "主标题", "subtext": "副标题"}))
pie.render('1.html')          #保存图片
pie.render_notebook()
```

【注】在 jupyter 中直接在页面显示图片: `chart_name.render_notebook()`

伪代码:

```
pie = (Pie()          #链式调用
      .add()          # bar 图则为 add_xaxis()和 add_yaxis()
      .set_global_opts(title_opts=opts.TitleOpts(title="主标题",
subtitle="副标题"))) #或者直接使用字典参数，如下:
      # .set_global_opts(title_opts={"text": "主标题", "subtext": "副标题"
      })
      )
pie.render('name.html')      #保存图片
#pie.render_notebook()      #在 jupyter 下使用
```

【注】在 jupyter 中直接在页面显示图片: `chart_name.render_notebook()`

由于 `pyrcharts` 作图步骤都一致，所以不需要每种作图都讲或者演示，可以只选讲 `Pie/Bar`、`Map/Geo`、`Graph/WordCloud` 三种类型即可。主要是讲解他们引用数据的形状。

《数据处理与分析（Python）》课程教案（16）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	3
教学单元（教学章、节或主题）：第 8 章 数据可视化					
目的、要求（分了解、理解、掌握三个层次） （1）学生能够学会 pyecharts 基本用法； （2）能够使用 pyecharts 实现地图的数据标注案例；					
重点 能够使用 pyecharts 实现地图的数据标注案例					
难点 能够使用 pyecharts 实现地图的数据标注案例					
教学步骤 知识点引入 → 讲解/讨论 → 知识点小结 → 作业/任务布置					
教具及教学手段 （如：举例讲解、多媒体讲解、模型讲解、实物讲解、挂图讲解、音像讲解、智慧教学工具使用等） 举例讲解、多媒体讲解、博思智慧教学工具使用相结合					
作业和思考题： 课前预习任务： 课后思考题： 课后作业/任务：					
教 学 后 记					

教 学 过 程	板 书 或 旁 注
---------	-----------

Pyecharts 地图

Pyecharts 的地图功能主要依靠 Geo 和 Map 两个类实现。其中 Geo 实现了一个地理坐标系，地图上的点可以利用经纬度向地图中插入，也可以获取地图上某一点的经纬度，实现地图上的标注功能主要依靠 Geo 类来实现。而 Map 功能类似于 Geo，但只有地图，没有坐标系，即地图上的点无法与经纬度进行转换。

1 地理坐标系（Geo）

Geo 在使用时需要调用以下模块。

```
from pyecharts import options as opts
```

```
from pyecharts.charts import Geo
```

```
from pyecharts.globals import ChartType, SymbolType
```

ChartType 是描述在地图上的标注形式，如 EFFECT_SCATTER、HEATMAP、LINES 等。

地图上显示的数据格式是二元列表，如[['name1', value1], ['name2', value2],...]。这里的 name 可以是省份、城市名称，在地图模型中已经加入了各个省份及城市的坐标点。

```
from pyecharts import options as opts
```

```
from pyecharts.charts import Geo
```

```
from pyecharts.globals import ChartType, SymbolType
```

```
ah_data=[['安庆市', 54], ['合肥市', 65], ['六安市', 76], ['马鞍山市', 64],  
         ['芜湖市', 35], ['池州市', 35], ['蚌埠市', 54], ['淮北市', 34],  
         ['淮南市', 56], ['黄山市', 87], ['阜阳市', 43], ['滁州市', 65],  
         ['宣城市', 47], ['亳州市', 45], ['宿州市', 23], ['铜陵市', 45],  
         ['潜山市', 51]] #假设 Geo 数据源中没有潜山市
```

#链式调用

```
anhui = (Geo()
```

```
    .add_schema(maptype="安徽")
```

```
    # 加入自定义的点
```

```
    .add_coordinate("潜山市", 116.53, 30.62)
```

```
    #添加数据
```

```
    .add("geo", ah_data,type_=ChartType.EFFECT_SCATTER)
```

```
    .set_series_opts(label_opts=opts.LabelOpts(is_show=True))
```

```
    .set_global_opts(visualmap_opts=opts.VisualMapOpts(is_pieewise=Tru
```

```
e),
```

```
                    title_opts=opts.TitleOpts(title="加入潜山市")))
```

#在 html(浏览器) 中渲染图表,即保存为 html 格式

```
anhui.render('anhui.html')
```

#在 Jupyter Notebook 中渲染图表

```
anhui.render_notebook('anhui.html')
```

2 地图（Map）

通过前面的 Geo，我们大概了解了地图标注的操作。Map 与 Geo 差别不大，通

过下面的代码，可以看出其操作相对较简单。

```
from pyecharts import options as opts
from pyecharts.charts import Map
```

#数据准备

```
provinces = ["广东", "北京", "上海", "新疆", "安徽", "山西", "湖南", "浙江", "江苏"]
pro_value = [54, 87, 56, 34, 98, 65, 45, 56, 78, 50]
pr_data = [list(z) for z in zip(provinces, pro_value)]
```

```
map = (
    Map()
    .add("商家 A", pr_data, "china")
    .set_global_opts(title_opts=opts.TitleOpts(title="Map-基本示例"))
)
```

```
map.render('map.html')
```

```
#map.render_notebook()
```

上面的代码基本与 Geo 相同，仅将 add_schema 项的地图显示范围参数移到了 add 项中。参数可选 world、china 及省市。

以上数据在地图显示中不明显，相关的省份没有颜色显示，可以增加省份颜色显示。

```
map_v = (
    Map()
    .add("商家 A", pr_data, "china")
    .set_global_opts(
        title_opts=opts.TitleOpts(title="Map-VisualMap（分段型）"),
        visualmap_opts=opts.VisualMapOpts(max_=200,
is_pieewise=True),
    )
)
```

```
map_v.render('map1.html')
```

```
map_v.render_notebook()
```

我们对数据进行改造，并按省份地图显示，代码如下，如图 8-18 所示。

```
ah_data=[['安庆市', 54], ['合肥市', 65], ['六安市', 76], ['马鞍山市', 64],
        ['芜湖市', 35], ['池州市', 35], ['蚌埠市', 54], ['淮北市', 34],
        ['淮南市', 56], ['黄山市', 87], ['阜阳市', 43], ['滁州市', 65],
        ['宣城市', 47], ['亳州市', 45], ['宿州市', 23], ['铜陵市', 45],
        ['潜山市', 51]] #假设 Geo 数据源中没有潜山市
```

```
map_v = (Map()
    .add("商家 A", ah_data, "安徽")
    .set_global_opts(
        title_opts=opts.TitleOpts(title="Map-VisualMap（省份）"),
        visualmap_opts=opts.VisualMapOpts(max_=200,
is_pieewise=True),
```

```
)  
)  
map_v.render('map2.html')  
#map_v.render_notebook()
```

Map-VisualMap (省份)

